

CS410

Parallel Computing: An Introduction

Milind Chabbi

Nikhil Hegde

Department of Computer Science and Engineering
IIT Dharwad

Slide adapted from John Mellor-Crummey's Comp422 @ Rice University

Course Information

- **Instructors:**
 - Milind Chabbi and Nikhil Hegde
 - {chabbi.milind, nikhilh}@iitdh.ac.in
 - email through google class room preferred
- **Time (tentative): Wednesday 8:30AM, Friday 9:30AM**
 - OR
 - In slot J when instructor Nikhil Hegde is teaching
- **Delivery mode:**
 - Hybrid:
 - Online via google class room (Milind Chabbi)
 - In class (Nikhil Hegde)
- **Teaching Assistants:**
 - Kumud Singh (cs24dp012)
 - TBD

Parallelism

- **Parallelism: simultaneous execution of multiple parts of a computation**
- **Reasons for applying parallelism**
 - **speed up: solve a problem faster**
 - **i.e. strong scaling**
 - **scale up: solve a larger problem**
 - **i.e. weak scaling**
 - **scale out: solve many problems**
 - **“... screen billions of small molecules against all the major non-structural proteins of SARS-CoV-19”**
- **CS410 focus: speed up - solving problems faster**

Units of Parallelism?

- **Terms**

- thread

- a sequence of instructions
 - managed by the OS or a runtime system

- process

- address space containing one or more threads

- **How big are the units of parallelism?**

- bit-level parallelism: operate on multiple bits in a word
 - vector instructions: operate on multiple independent bytes or words
 - instruction-level parallelism: > 1 instructions from a thread at once
 - multiple threads in a process
 - multiple processes

- **CS410 focus: thread and process parallelism**

Course Objectives

- **Learn fundamentals of parallel computing**
 - principles of parallel algorithm design
 - programming models and methods
 - parallel computer architectures
 - parallel algorithms
 - modeling and analysis of parallel programs and systems
- **Develop skill writing parallel programs**
 - programming assignments
- **Develop skill analyzing parallel computing problems**
 - solving problems posed in class

Recommended Books

- **Introduction to Parallel Computing, 2nd Ed, Ananth Grama, Anshul Gupta, George Karypis, Vipin Kumar (2003)**
- **Using OpenMP: Portable Shared Memory Parallel Programming - Barbara Chapman, Gabriele Jost, Ruud van der Pas (2008) [\[PDF online\]](#)**
- **Using MPI: Portable Parallel Programming with the Message-Passing Interface, 3rd Ed - William Gropp, Ewing Lusk, Anthony Skjellum (2014)**
- **Programming Massively Parallel Processors: A Hands-on Approach, 3rd Ed. - David B. Kirk, Wen-mei W. Hwu (2016) [\[PDF of 2nd edition available online\]](#)**

Topics (Part 1)

- **Introduction**
- **Principles of parallel algorithm design**
 - decomposition techniques
 - mapping & scheduling computation
 - templates
- **Programming shared-address space systems**
 - Cilk
 - OpenMP
 - Pthreads
 - synchronization
- **Parallel computer architectures**
 - shared memory systems and cache coherence
 - distributed-memory systems
 - interconnection networks and routing

Topics (Part 2)

- **Programming scalable systems**
 - message passing: MPI
 - global address space languages
- **Collective communication**
- **Analytical modeling of program performance**
 - speedup, efficiency, scalability, cost optimality, isoefficiency
- **Parallel algorithms**
 - non-numerical algorithms: sorting, graphs
 - numerical algorithms: dense and sparse matrix algorithms
- **Performance measurement and analysis of parallel programs**
- **GPU Programming with CUDA**

Prerequisites

- **Programming in C, C++, or similar**
- **Basics of data structures**
- **Basics of machine architecture**
- **See the instructor(s) if you have concerns**

Motivations for Parallel Computing

- **Technology push**
- **Application pull**

The Rise of Multicore Processors

Advance of Semiconductors: “Moore’s Law”

Gordon Moore, Founder of Intel

- **1965: since the integrated circuit was invented, the number of transistors in an integrated circuit has roughly doubled every year; this trend would continue for the foreseeable future**
- **1975: revised - circuit complexity doubles every two years**

Gordon Moore



Moore in 1978

Born

Gordon Earle Moore

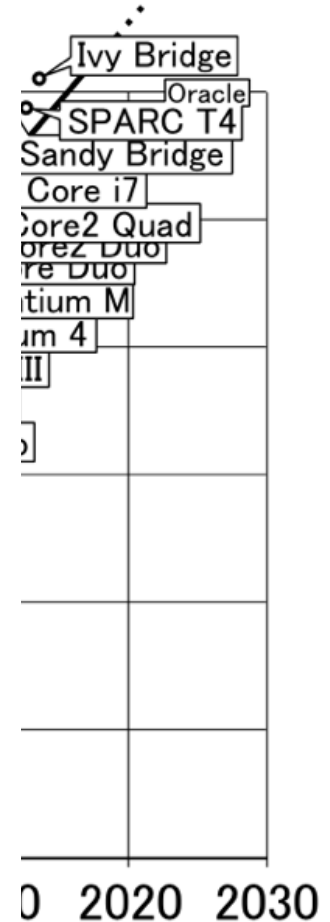
January 3, 1929

Pescadero, California, U.S.^[3]

Died

March 24, 2023 (aged 94)

Waimea, Hawaii, U.S.

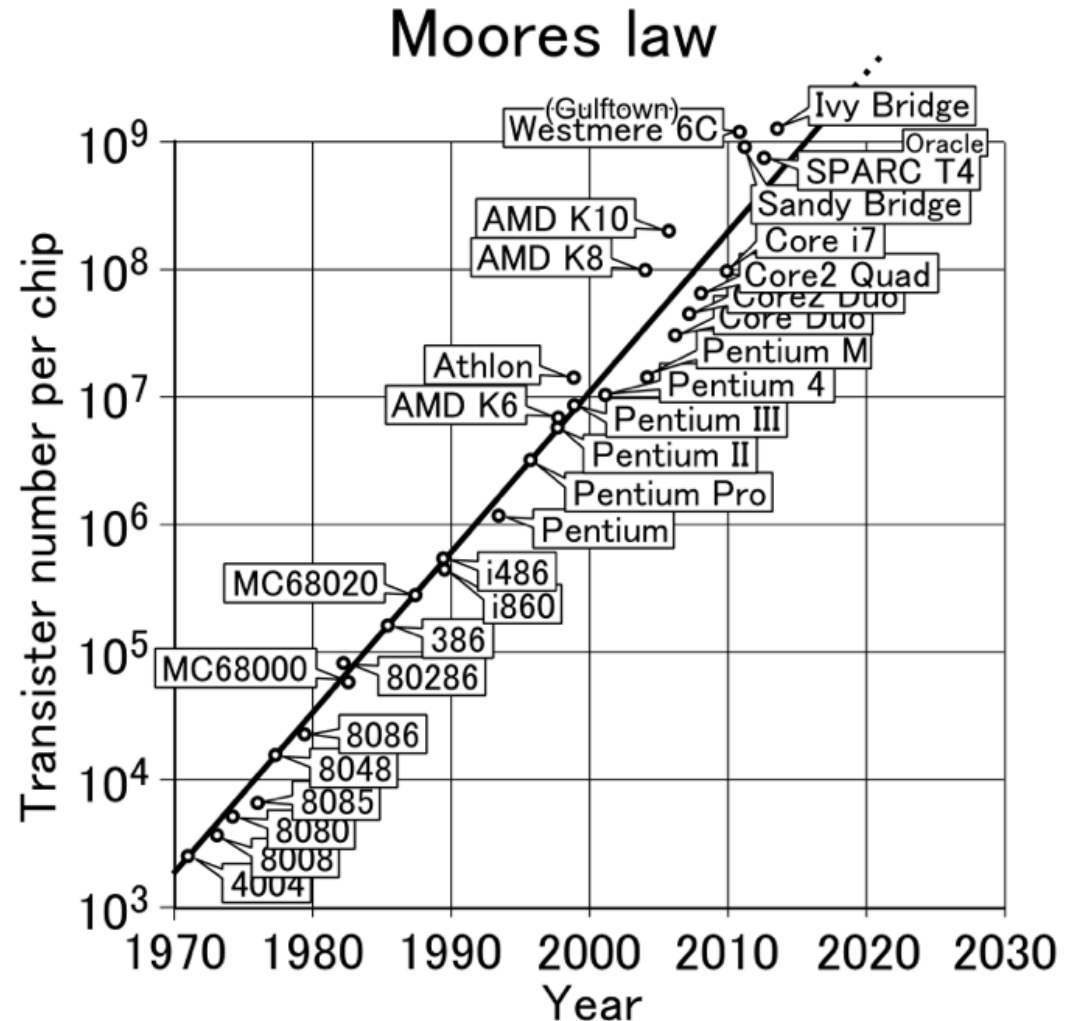


dia Commons

Advance of Semiconductors: “Moore’s Law”

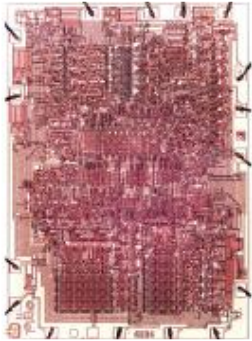
Gordon Moore, Founder of Intel

- **1965:** since the integrated circuit was invented, the number of transistors in an integrated circuit has roughly doubled every year; this trend would continue for the foreseeable future
- **1975:** revised - circuit complexity doubles every two years

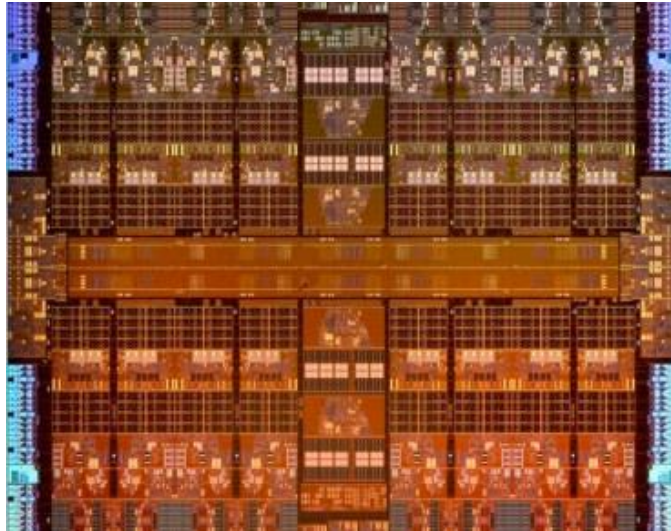


By shigeru23 CC BY-SA 3.0, via Wikimedia Commons

Evolution of Microprocessors 1971-2017



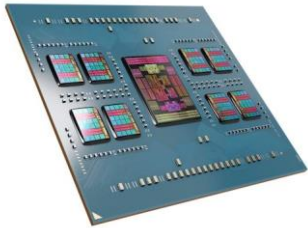
Intel 4004, 1971
1 core, no cache
23K transistors



Oracle M7, 2015
32 cores, 64MB cache
10B transistors



Nvidia Ampere A100, 2022
6912 CUDA cores
54B transistors



AMD [EPYC Bergamo \(4th gen/97X4 series\)](#) 9-chip
module (up to 128 cores and
256 MB (L3) + 128 MB (L2)
cache), 2023
82B transistors



[Apple M2 Ultra](#) (two M2 Max dies), 2023
132B transistors

Cerebras Wafer Scale Engine 2

Cerebras WSE-2 The Largest Chip Ever Built

46,225	mm ² silicon
2.6	Trillion transistors
850,000	AI optimized cores
40	Gigabytes on-chip memory
20	Petabyte/s memory bandwidth
220	Petabit/s fabric bandwidth
7nm	Process technology at TSMC

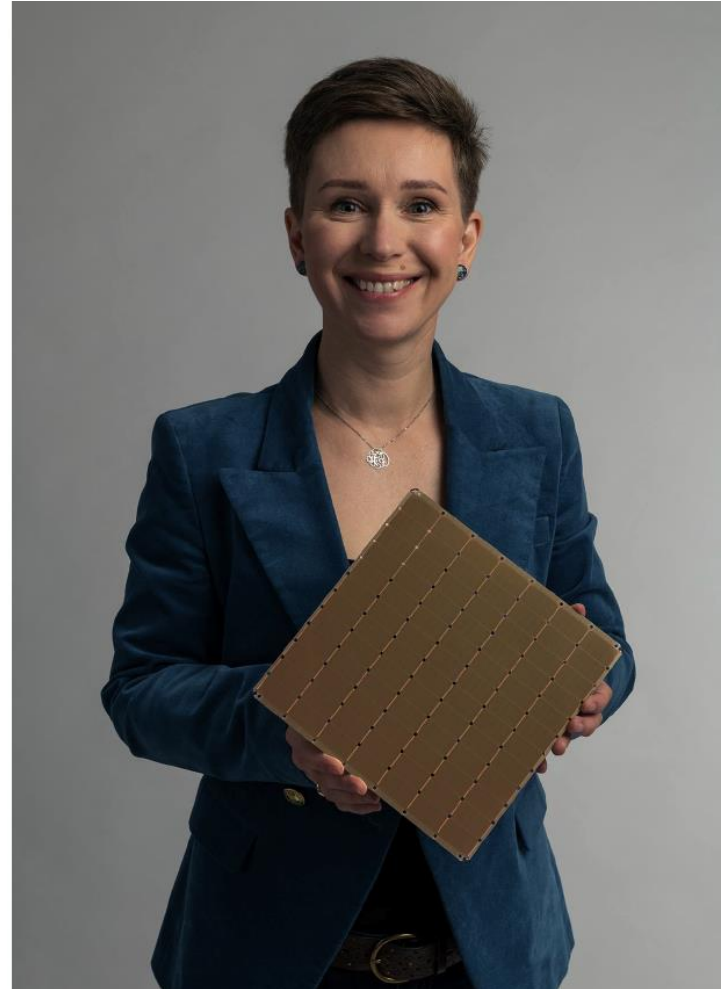


Figure Credit: Sean Lie, Multi-Million Core, Multi-Wafer AI Cluster. Hot Chips 33. August 22-24, 2021. (Virtual Event)

Leveraging Moore's Law Trends

From increasing transistor count to performance

- More transistors = opportunities for exploiting parallelism
- Parallelism in a CPU core
 - implicit parallelism: invisible to the programmer
 - pipelined execution of instructions
 - multiple functional units for multiple independent pipelines
 - explicit parallelism
 - long instruction words (VLIW)
 - bundles of independent instructions that can be issued together
 - e.g., Intel Itanium processor 2000-2017
 - SIMD processor extensions up to 512 bits wide (AVX512)
 - integer, floating point, complex data
 - operations on up to 16 32-bit data items per instruction

Microprocessor Architecture (Mid 90's)

- **Superscalar (SS) designs were the state of the art**
 - multiple functional units (e.g., int, float, branch, load/store)
 - multiple instruction issue
 - dynamic scheduling: HW tracks instruction dependencies
 - speculative execution: look past predicted branches
 - non-blocking caches: multiple outstanding memory operations
- **Apparent path to higher performance?**
 - wider instruction issue
 - support for more speculation

Trouble on the Horizon

Increasing issue width provides diminishing returns

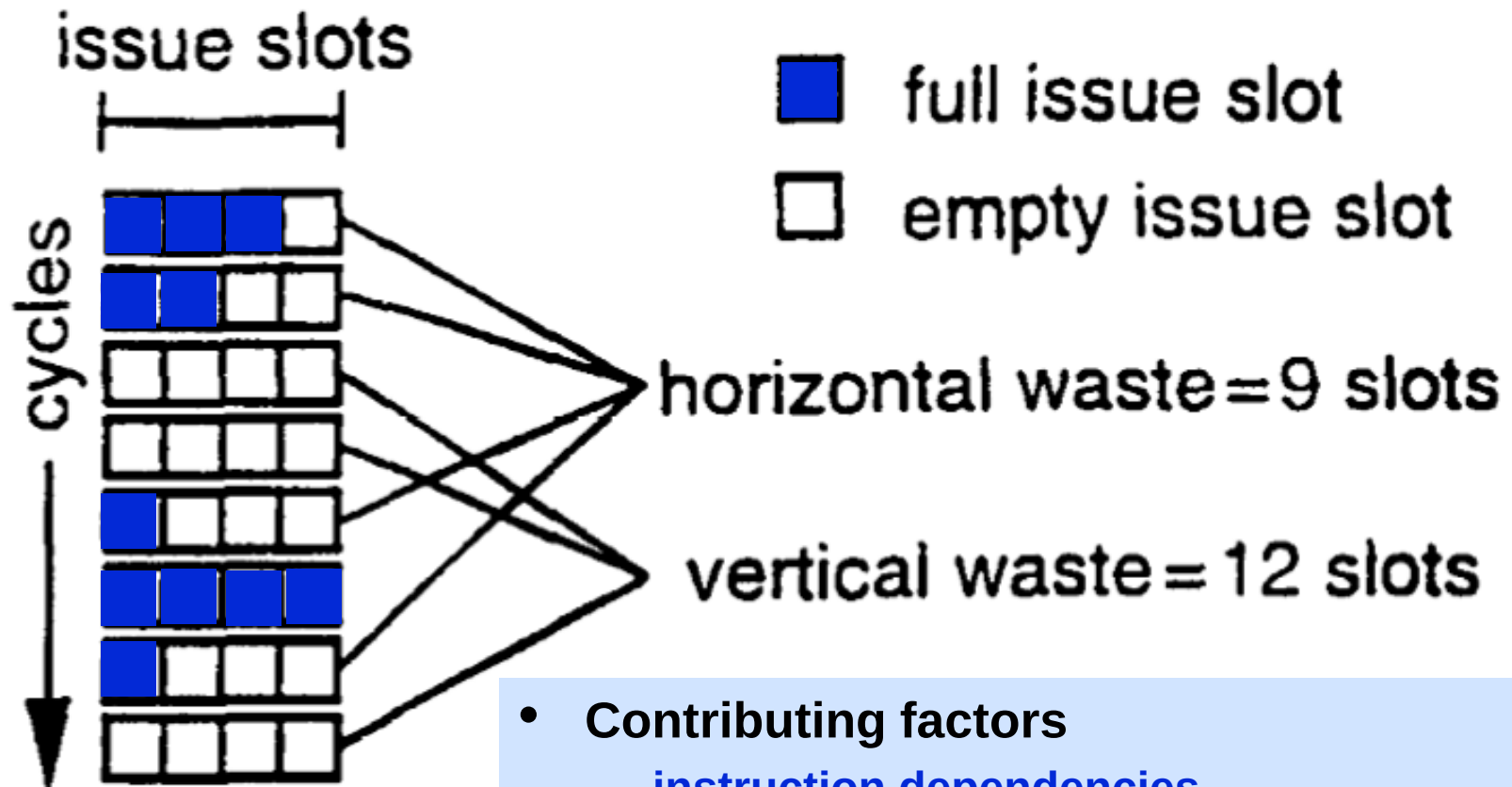
Two factors¹

- **Fundamental circuit limitations**
 - delays as issue queues and multi-port register files
 - increasing delays limit performance returns from wider issue
- **Limited amount of instruction-level parallelism**
 - inefficient for programs with difficult-to-predict branches

¹[The case for a single-chip multiprocessor](#), K. Olukotun, B. Nayfeh, L. Hammond, K. Wilson, and K. Chang, ASPLOS-VII, 1996.

Instruction-level Parallelism Concerns

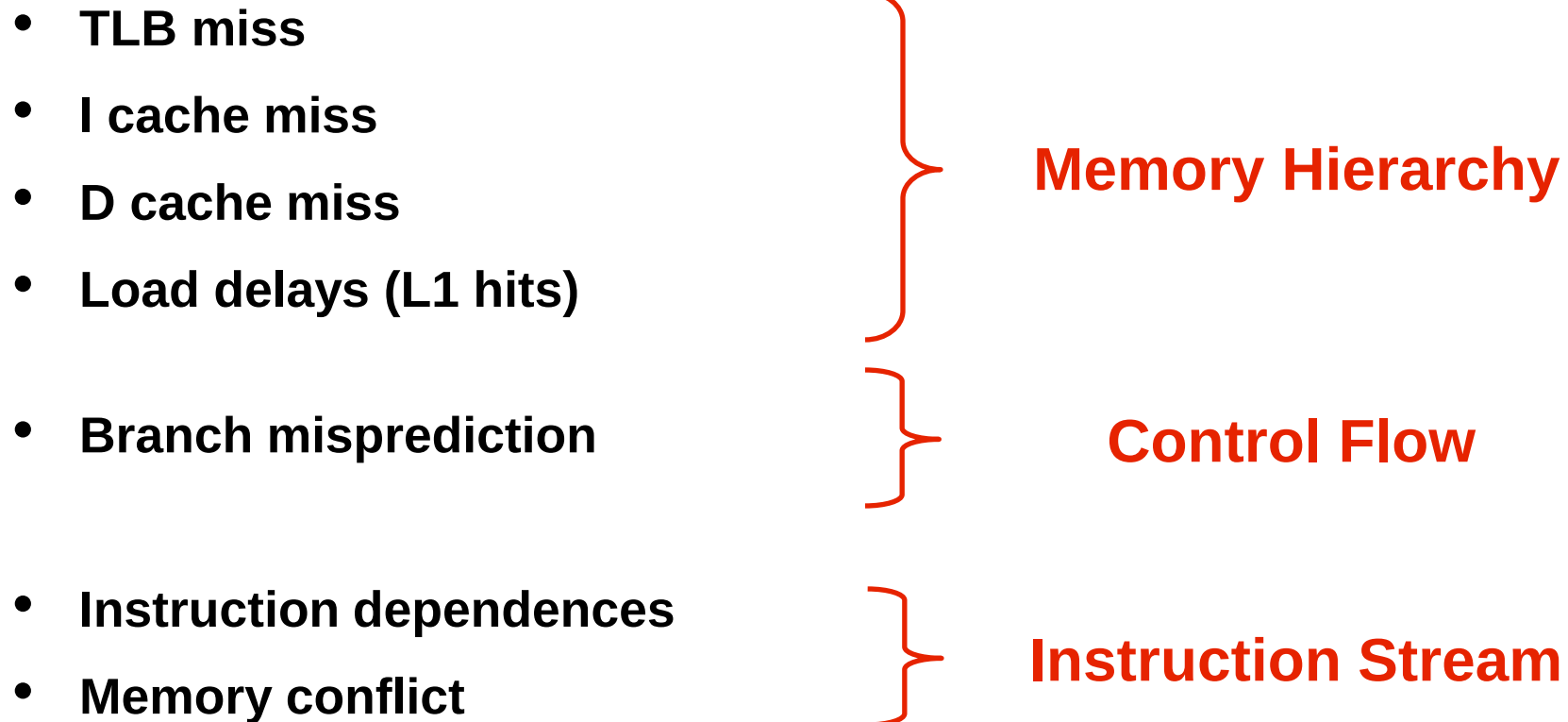
Issue Waste



- Contributing factors
 - instruction dependencies
 - long-latency operations within a thread

Some Sources of Wasted Issue Slots

- TLB miss
- I cache miss
- D cache miss
- Load delays (L1 hits)
- Branch misprediction
- Instruction dependences
- Memory conflict



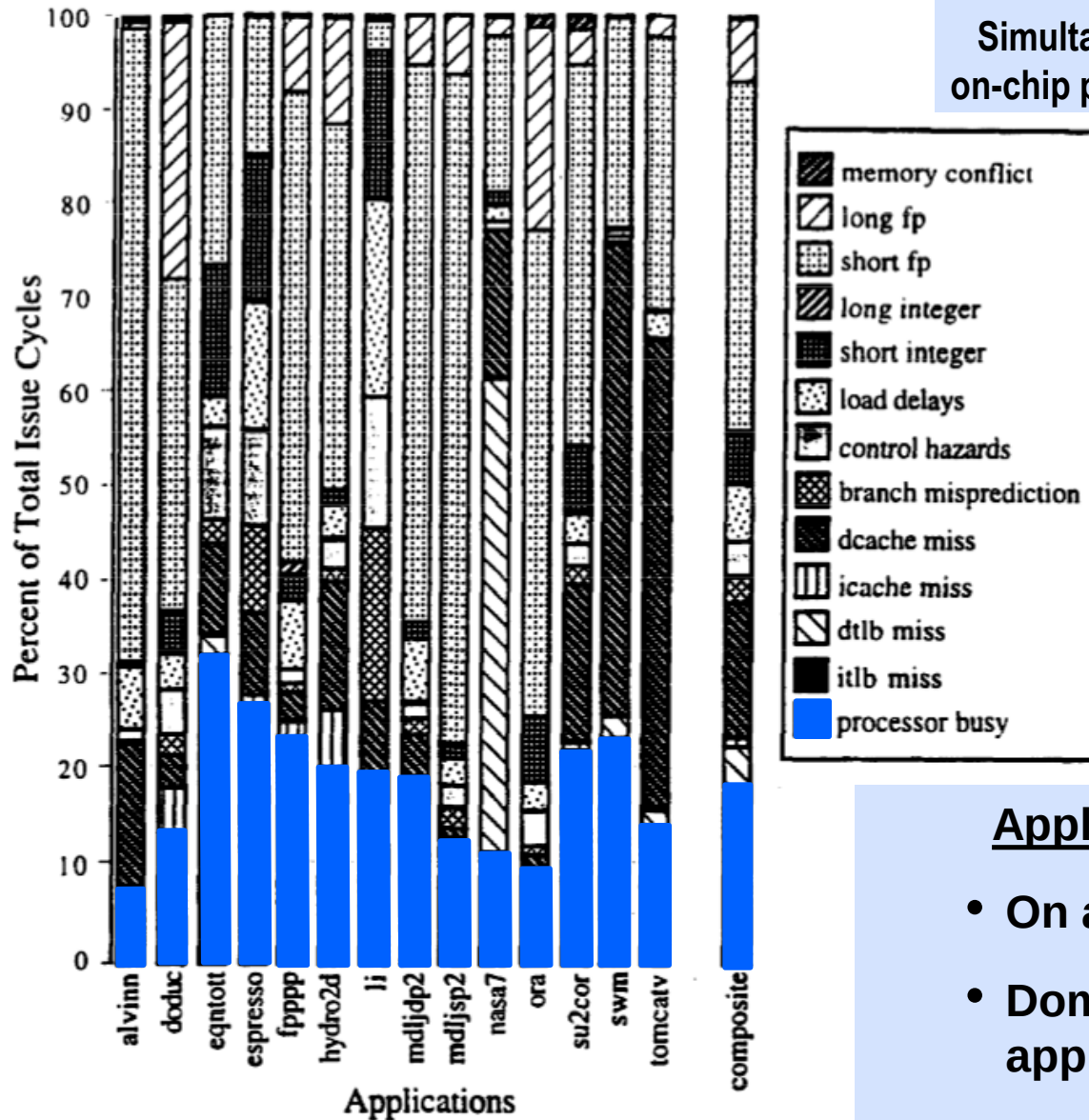
Memory Hierarchy

The diagram illustrates the sources of wasted issue slots in a processor. On the left, a list of seven sources is provided. On the right, three categories are listed, each preceded by a red curly brace that groups specific sources from the list. The first category, 'Memory Hierarchy', groups 'TLB miss', 'I cache miss', 'D cache miss', and 'Load delays (L1 hits)'. The second category, 'Control Flow', groups 'Branch misprediction'. The third category, 'Instruction Stream', groups 'Instruction dependences' and 'Memory conflict'.

Control Flow

Instruction Stream

Simulations of 8-issue Superscalar



Simultaneous multithreading: maximizing on-chip parallelism, Tullsen et. al. ISCA, 1995.

Summary:
Highly underutilized

Applications: most of SPEC92

- On average < 1.5 IPC (19%)
- Dominant waste differs by application
- Short FP dependences: 37%

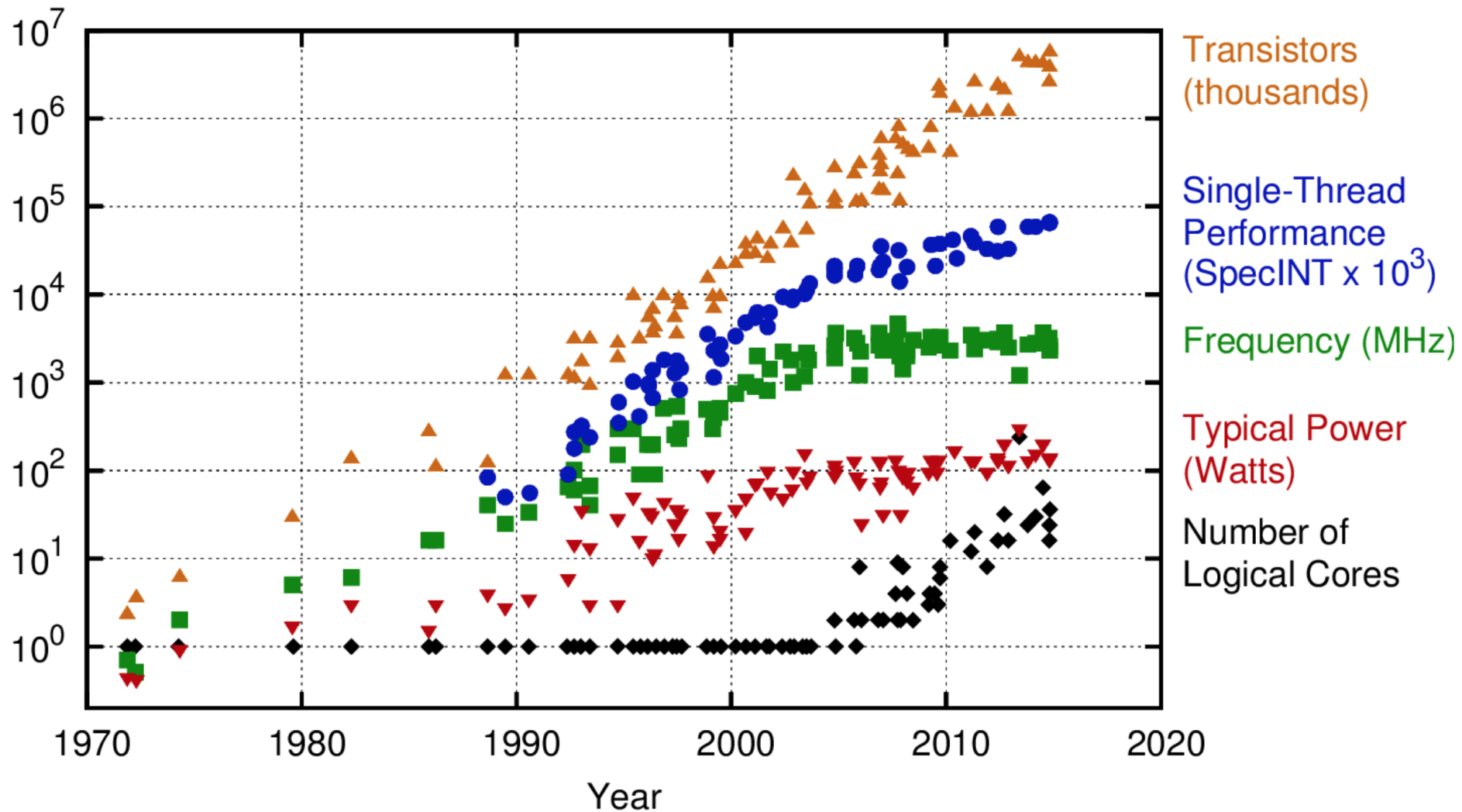
Power and Heat Stall Clock Frequencies

May 17, 2004 ... Intel, the world's largest chip maker, publicly acknowledged that it had hit a "thermal wall" on its microprocessor line. As a result, the company is changing its product strategy and disbanding one of its most advanced design groups. Intel also said that it would abandon two advanced chip development projects ...

Now, Intel is embarked on a course already adopted by some of its major rivals: obtaining more computing power by stamping multiple processors on a single chip rather than straining to increase the speed of a single processor ... Intel's decision to change course and embrace a "dual core" processor structure shows the challenge of overcoming the effects of heat generated by the constant on-off movement of tiny switches in modern computers ... some analysts and former Intel designers said that *Intel was coming to terms with escalating heat problems so severe they threatened to cause its chips to fracture at extreme temperatures...*

New York Times

Technology Trends

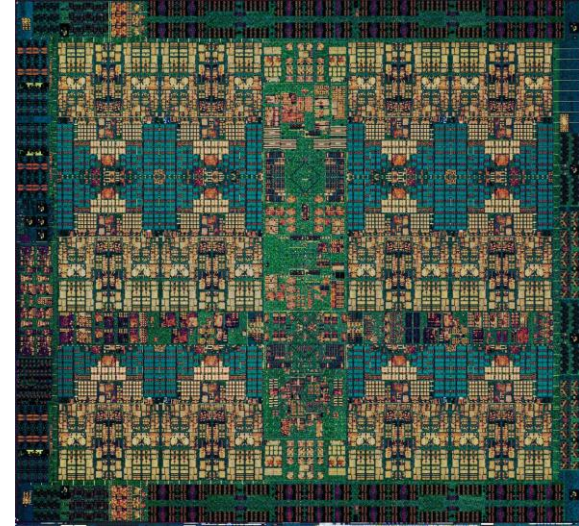


Original data up to the year 2010 collected and plotted by M. Horowitz, F. Labonte, O. Shacham, K. Olukotun, L. Hammond, and C. Batten
New plot and data collected for 2010-2015 by K. Rupp

Figure credit: Karl Rupp. <https://www.karlrupp.net/wp-content/uploads/2015/06/40-years-processor-trend.png>.

Recent Multicore Processors

- **2023: AMD Bergamo**
— 128 cores, 2-way SMT, 256MB L3 cache
- **2019: AMD EPYC 7742**
— 64 cores; 2-way SMT; 256MB cache
- **2017: IBM Power9**
— 24 cores; 4-way SMT; 120MB cache
- **2016: Intel Knight's Landing**
— 72 cores; 4-way SMT; 36MB cache
- **2015: Oracle SPARC M7**
— 32 cores; 8-way SMT; 64MB cache



IBM Power9

<https://en.wikichip.org/wiki/ibm/microarchitectures/power9>

Application Pull

- **Complex problems require computation on large-scale data**
- **Sufficient performance is available only through massive parallelism**

Computing and Science

“Computational modeling and simulation are among the most significant developments in the practice of scientific inquiry in the 20th century. Within the last two decades, scientific computing has become an important contributor to all scientific disciplines. It is particularly important for the solution of research problems that are insoluble by traditional scientific theoretical and experimental approaches, hazardous to study in the laboratory, or time consuming or expensive to solve by traditional means”

— **“Scientific Discovery through Advanced Computing”**
DOE Office of Science, 2000